

Classes and Abstract Classes

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Classes in TypeScript

```
class Point {  
  x: number;  
  y: number;  
  constructor(x: number, y: number) {  
    this.x = x;  
    this.y = y;  
  }  
  distanceFromOrigin(): number {  
    return Math.sqrt(this.x ** 2 + this.y ** 2);  
  }  
}  
new Point(3, 4).distanceFromOrigin(); // 5
```

- TS requires you to **declare fields** and checks the constructor initializes them

Parameter Properties

```
class Point {  
  constructor(public x: number, public y: number) {}  
  // Declares x and y AND assigns this.x = x, this.y = y  
}  
  
const p = new Point(3, 4);  
p.x; // 3
```

- Put an access modifier on a constructor parameter, TS creates and assigns the field
- A shorthand for the very common declare-then-assign pattern

Access Modifiers

```
class BankAccount {  
  private balance = 0;  
  deposit(amount: number): void { this.balance += amount; }  
  getBalance(): number { return this.balance; }  
}  
  
const acct = new BankAccount();  
acct.deposit(100);  
acct.balance; // Error: 'balance' is private
```

- **public** (default) everywhere · **private** this class only
- **protected** this class + subclasses · **readonly** set once in constructor

private vs #private

```
class A {  
    private secret = 1;    // TypeScript only - erased at compile time  
    #realSecret = 2;       // JavaScript - enforced at RUNTIME  
}  
  
const a = new A() as any;  
a.secret;    // 1 - the check was compile-time only!  
a["#realSecret"]; // undefined - genuinely unreachable
```

- `private` stops *your code* compiling; `#` stops *any code* at runtime
- Use `#` when the guarantee has to survive the type erasure

Implementing an Interface

```
interface Shape {  
  area(): number;  
  name: string;  
}  
  
class Circle implements Shape {  
  name = "circle";  
  constructor(private radius: number) {}  
  area(): number { return Math.PI * this.radius ** 2; }  
}
```

- **implements** makes a class satisfy an interface's contract
- TS reports any missing member; a class can implement **multiple** interfaces
- **implements** only checks the shape; it inherits no code

Abstract Classes

```
abstract class Shape {  
  abstract area(): number;      // no body - subclasses MUST implement  
  describe(): string {          // concrete, shared method  
    return `This shape has area ${this.area()}`;  
  }  
}  
  
new Shape();  
// Error: Cannot create an instance of an abstract class.
```

- An **abstract class** cannot be instantiated; it exists to be extended

Abstract Methods

```
class Rectangle extends Shape {  
  constructor(private width: number, private height: number) {  
    super();  
  }  
  area(): number { return this.width * this.height; } // required  
}  
  
const r = new Rectangle(3, 4);  
r.area();      // 12  
r.describe();  // "This shape has area 12" (inherited)
```

- Every concrete subclass **must** implement each abstract method

Abstract Class vs Interface

- **Interface**: purely a shape, no implementation; a class can implement **many**
- **Abstract class**: can hold **shared code** *and* enforce abstract members, but a class extends only **one**
- Need to **share code** among related classes? Use an **abstract class**
- Only need to **guarantee a shape**? Use an **interface**
- An interface disappears at compile time; an abstract class is real JS output

What TypeScript Is Typing

Class Anatomy

Where each part actually lives: fields on the instance, methods on the prototype, statics on the class itself.

```
class Animal {
  static kingdom = 'Animalia';
  #secret = 'hidden';
  constructor(name) { this.name = name; }
  speak() { return this.name + ' makes a sound'; }
  get label() { return this.name.toUpperCase(); }
}
class Dog extends Animal {
  constructor(name, breed) {
    super(name);
    this.breed = breed;
  }
  speak() { return super.speak() + ' (a bark)'; }
}
```

class is syntax over the prototype chain, not a new object model: the same lookup rules apply. Two things it does add: #private fields are enforced by the language (unlike a leading underscore), and the whole class body runs in strict mode.

On the INSTANCE

name, breed
#secret
One copy per object. Anything assigned to this in the constructor.

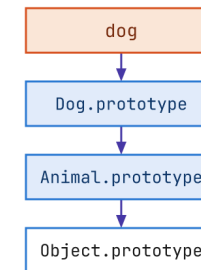
On the PROTOTYPE

speak(), get label
ONE copy, shared by every instance. This is why methods cost nothing per object.

On the CLASS

Animal.kingdom
Statics belong to the class, not to any instance. dog.kingdom is undefined.

What extends actually does



extends just links these. super(name) runs the parent constructor, and you must call it before touching this.

- The same instance / prototype / class layout, now checked
- `abstract` exists only at compile time; it is erased like every other type